



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Длинная арифметика

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Написание собственной структуры данных	5
2.2 Сложение двух чисел	5
2.3 Вычитание двух чисел	6
2.4 Операторы сравнения	6
2.5 Оператор ввода	7
2.6 Оператор вывода	7
3 Технологическая часть	8
4 Исследовательская часть	22
Заключение	26
Список используемых источников	27

Введение

Цель лабораторной работы: создать собственную структуру данных.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать троичные числа.
2. Разработать структуру данных для хранения целых чисел с повышением ёмкости хранения по сравнению с базовым типом данных.
3. Реализовать 6 операторов сравнения, операторы присваивания (копии значения), сложения и вычитания двух структур длинных чисел.
4. Выполнить тестирование.

Согласно варианту, требуется разработать структуру хранения троичных чисел.

1 Аналитическая часть

Длинная арифметика — арифметика, которая относится к арифметическим операциям над числами, превышающими предельные значения стандартных числовых типов данных (например, “int”, “long”, “double” и т.д.), либо к работе с числами, которые требуют более высокой точности или специфических условий хранения.

Троичные числа [1] — это числа, которые используют систему счисления с основанием 3. В отличие от привычной для нас десятичной системы счисления (основание 10), где используются цифры от 0 до 9, в троичной системе используются только три цифры: 0, 1 и 2.

Емкость хранения — количество данных или информации, которое может быть сохранено или содержано в определенной системе, устройстве, структуре данных или носителе информации.

Тип данных — определяет характеристики данных, их формат, допустимый диапазон значений и операции, которые можно выполнять над этими данными.

2 Конструкторская часть

2.1 Написание собственной структуры данных

С помощью класса “LongTritNumber”, который описывает структуру данных для хранения целых чисел с повышенной емкостью хранения по сравнению с базовым типом данных, реализовать арифметику и сравнения для чисел, представленных троичными цифрами (False, Unknown, True) с емкостью хранения в 8 тритов. Требуется реализовать следующие методы и перегруженные операторы.

- Метод “addTrits” — метод, который складывает два трита и возвращает сумму.
- Метод “calculateCarry” — метод, который вычисляет перенос при сложении троичных чисел.
- Метод “subtractTrits” — метод, который вычитает два трита и возвращает разность.
- Метод “calculateBorrow” — метод, который вычисляет перенос при вычитании троичных чисел.
- “operator[]” — перегруженный оператор для доступа к тритам по индексу.
- “operator=” — перегруженный оператор для операции присваивания.
- Операторы сравнения (“operator==”, “operator!=”, “operator<”, “operator>”, “operator<=”, “operator>=”) — реализуют сравнение двух чисел по их тритам.
- Операторы арифметики (“operator+”, “operator-”) — реализуют сложение и вычитание двух чисел.
- Дружественная функция-оператор “operator«” — перегруженный оператор для вывода числа в виде строки троичных значений.
- Дружественная функция-оператор “operator»” — перегруженный оператор для ввода числа с клавиатуры.

2.2 Сложение двух чисел

1. Создается новый объект типа “LongTritNumber”, предназначенный для хранения результата сложения.
2. Инициализируется переменная “carry” с начальным значением False.
3. Итерация по тритам.
 - Происходит итерация через триты чисел с конца к началу.

- Выполняется вызов вспомогательной функции “addTrits” для сложения тритов и получения суммы и значения переноса.
 - Результат сложения помещается в соответствующий разряд нового объекта типа “LongTritNumber”.
 - Перенос сохраняется для использования на следующей итерации.
4. Возвращается объект типа “LongTritNumber”, содержащий результат сложения.

2.3 Вычитание двух чисел

1. Создается новый объект типа “LongTritNumber”, предназначенный для хранения результата вычитания.
2. Инициализируется переменная “borrow” с начальным значением False.
3. Итерация по тритам.
 - Происходит итерация через триты чисел с конца к началу.
 - Выполняется вызов вспомогательной функции “subtractTrits” для вычитания тритов и получения разности и значения переноса.
 - Результат вычитания помещается в соответствующий разряд нового объекта типа “LongTritNumber”.
 - Перенос сохраняется для использования на следующей итерации.
4. Возвращается объект типа “LongTritNumber”, содержащий результат вычитания.

2.4 Операторы сравнения

1. Оператор равенства.
 - Пройти по всем тритам в обоих числах, сравнивая их между собой по позициям.
 - Если все триты в соответствующих позициях равны, возвращает “true”. Если хотя бы один трит не совпадает, возвращает “false”.
2. Оператор неравенства.
 - Пройти по всем тритам в обоих числах, сравнивая их между собой по позициям.
 - Если все триты в соответствующих позициях равны, возвращает “false”. Если хотя бы один трит не совпадает, возвращает “true”.
3. Оператор сравнения по отношению.
 - Пройти по тритам чисел, начиная с наиболее значимых разрядов

- Если найдено отличие, операторы возвращают “true” или “false” в соответствии с результатом сравнения.
- Если все триты в соответствующих позициях равны, операторы могут продолжить сравнение следующих тритов, пока не будет найдено отличие или не закончатся триты для сравнения.

2.5 Оператор ввода

1. Для каждого элемента в векторе “trits” (переменная, хранящая триты объекта “LongTritNumber”) выполняется итерация. При каждой итерации запрашивается у пользователя значение для трита с помощью стандартного ввода.
2. После ввода значения проверяется, находится ли оно в допустимом диапазоне (0, 1 или 2). Это происходит с помощью проверки условия “if”. Если введенное значение не соответствует триту (0, 1 или 2), выводится сообщение о некорректном вводе и оставшиеся незаполненные элементы вектора автоматически заполняются значениями 0. Если значение корректно, оно сохраняется в текущий трит в векторе “trits”.
3. После завершения итерации по всем тритам в объекте “LongTritNumber” функция возвращает поток ввода.

2.6 Оператор вывода

1. Для каждого элемента в векторе “trits” (переменная, хранящая триты объекта “LongTritNumber”) выполняется итерация. При каждой итерации определяется значение трита и выводится соответствующее ему текстовое представление в поток вывода.
2. После завершения итерации по всем тритам в объекте “LongTritNumber” функция возвращает поток вывода.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<vector>`. Она представляет собой динамический массив, способный изменять свой размер в процессе выполнения программы.

Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для реализации стека

```
//#include <iostream>
#include <vector>
//
//using namespace std;
//
//class LongTritNumber {
//public:
//    enum class Trit {False = 0, Unknown = 1, True = 2 };
//    static const int Capacity = 8;
//
//    LongTritNumber() {
//        trits.resize(Capacity, Trit::False);
//    }
//
//    const Trit& operator[](size_t index) const {
//        return trits[index];
//    }
//
//    LongTritNumber& operator=(const LongTritNumber& other) {
//        if (this != &other) {
//            for (size_t i = 0; i < Capacity; ++i) {
//                trits[i] = other.trits[i];
//            }
//        }
//        return *this;
//    }
//
//    bool operator==(const LongTritNumber& other) const {
//        for (size_t i = 0; i < Capacity; ++i) {
//            if (trits[i] != other.trits[i]) {
//                return false;
//            }
//        }
//    }
//}
```



```

//          return true;
//      }
//
//      bool operator!=(const LongTritNumber& other) const {
//          for (size_t i = 0; i < Capacity; ++i) {
//              if (trits[i] == other.trits[i]) {
//                  return true;
//              }
//          }
//          return true;
//      }
//
//      LongTritNumber operator+(const LongTritNumber& other) const {
//          LongTritNumber result;
//          Trit carry = Trit::False;
//
//          for (int i = Capacity - 1; i >= 0; --i) {
//              Trit sum = addTrits(trits[i], other.trits[i],
//          carry);
//              result.trits[i] = sum;
//              carry = calculateCarry(trits[i], other.trits[i],
//          carry);
//          }
//
//          return result;
//      }
//
//      LongTritNumber operator-(const LongTritNumber& other) const {
//          LongTritNumber result;
//          Trit borrow = Trit::False;
//
//          for (int i = Capacity - 1; i >= 0; --i) {
//              Trit diff = subtractTrits(trits[i], other.trits[i],
//          ], borrow);
//              result.trits[i] = diff;
//              borrow = calculateBorrow(trits[i], other.trits[i],
//          ], borrow);
//          }
//
//          return result;
//      }
//

```

```

//      bool operator<(const LongTritNumber& other) const {
//          for (size_t i = 0; i < Capacity; ++i) {
//              if (trits[i] < other.trits[i]) {
//                  return true;
//              }
//              else if (trits[i] > other.trits[i]) {
//                  return false;
//              }
//          }
//          return false;
//      }
//
//      bool operator>(const LongTritNumber& other) const {
//          for (size_t i = 0; i < Capacity; ++i) {
//              if (trits[i] > other.trits[i]) {
//                  return true;
//              }
//              else if (trits[i] < other.trits[i]) {
//                  return false;
//              }
//          }
//          return false;
//      }
//
//      bool operator<=(const LongTritNumber& other) const {
//          for (size_t i = 0; i < Capacity; ++i) {
//              if (trits[i] <= other.trits[i]) {
//                  return true;
//              }
//              else if (trits[i] >= other.trits[i]) {
//                  return false;
//              }
//          }
//          return false;
//      }
//
//      bool operator>=(const LongTritNumber& other) const {
//          for (size_t i = 0; i < Capacity; ++i) {
//              if (trits[i] >= other.trits[i]) {
//                  return true;
//              }
//              else if (trits[i] <= other.trits[i]) {

```

```

//                                     return false;
//                                     }
//                                     }
//                                     return false;
//                                     }
//
//     friend ostream& operator<<(ostream& os, const LongTritNumber&
number) {
//         for (const auto& trit : number.trits) {
//             switch (trit) {
//                 case Trit::False:
//                     os << "False";
//                     break;
//                 case Trit::Unknown:
//                     os << "Unknown";
//                     break;
//                 case Trit::True:
//                     os << "True";
//                     break;
//             }
//             os << " ";
//         }
//         return os;
//     }
//
//     friend istream& operator>>(istream& is, LongTritNumber& number) {
//         for (auto& trit : number.trits) {
//             int userInput;
//             cout << Введите" значения трита (0 - False, 1 -
Unknown, 2 - True): ";
//             is >> userInput;
//
//             if (userInput >= 0 && userInput <= 2) {
//                 trit = static_cast<Trit>(userInput);
//             }
//             else {
//                 cout << Некорректный" ввод,
используйте значения от 0 до 2." << endl;
//                 return is;
//             }
//         }
//         return is;

```

```

//      }
//
//private:
//      vector<Trit> trits;
//
//      Trit addTrits(Trit a, Trit b, Trit& carry) const {
//          int sum = static_cast<int>(a) + static_cast<int>(b);
//          carry = static_cast<Trit>(sum / 3);
//          return static_cast<Trit>(sum % 3);
//      }
//
//      Trit calculateCarry(Trit a, Trit b, Trit carry) const {
//          int sum = static_cast<int>(a) + static_cast<int>(b) +
static_cast<int>(carry);
//          return static_cast<Trit>(sum / 3); // 1 если sum >= 3,
иначе 0
//      }
//
//      Trit subtractTrits(Trit a, Trit b, Trit& borrow) const {
//          int diff = static_cast<int>(a) - static_cast<int>(b) -
static_cast<int>(borrow);
//          if (diff < 0) {
//              diff += 3; // Коррекция для отрицательных значений
//              borrow = Trit::True;
//          }
//          else {
//              borrow = Trit::False;
//          }
//          return static_cast<Trit>(diff);
//      }
//
//      Trit calculateBorrow(Trit a, Trit b, Trit borrow) const {
//          int diff = static_cast<int>(a) - static_cast<int>(b) -
static_cast<int>(borrow);
//          return (diff < 0) ? Trit::True : Trit::False; // 1 если
diff < 0, иначе 0
//      }
//};
//
//int main() {
//    setlocale(0, "");
//    LongTritNumber a, b;

```

```

//
//      int otr1 = 0;
//      cout << "Число" аположительноеилиотрицательное      ? (0 - полож. / 1 -
отриц)";
//      cin >> otr1;
//      cout << "Введите" значениядляпеременной      a:" << endl;
//      cin >> a;
//
//      int otr2 = 0;
//      cout << "Число" b положительноеилиотрицательное      ? (0 - полож. / 1 -
отриц)";
//      cin >> otr2;
//      cout << "Введите" значениядляпеременной      b:" << endl;
//      cin >> b;
//
//      LongTritNumber c = a + b;
//      LongTritNumber d = a - b;
//
//      cout << "Результат" сложения: " << c << endl;
//      cout << "Результат" вычитания: " << d << endl;
//
//      if (a == b) {
//          cout << "a равно b" << endl;
//      }
//      else {
//          cout << "a не равно b" << endl;
//      }
//
//      if (a != b) {
//          cout << "a не равно b" << endl;
//      }
//      else {
//          cout << "a равно b" << endl;
//      }
//
//      if (a < b) {
//          cout << "a меньше b" << endl;
//      }
//      else {
//          cout << "a не меньше b" << endl;
//      }
//

```

```

//      if (a > b) {
//          cout << "a больше b" << endl;
//      }
//      else {
//          cout << "a не больше b" << endl;
//      }
//
//      if (a <= b) {
//          cout << "a меньше или равно b" << endl;
//      }
//      else {
//          cout << "a не меньше или равно b" << endl;
//      }
//
//      if (a >= b) {
//          cout << "a больше или равно b" << endl;
//      }
//      else {
//          cout << "a не больше или равно b" << endl;
//      }
//
//      return 0;
//}
//

#include <iostream>
#include <vector>

using namespace std;

class LongTritNumber {
public:
    enum class Trit { False = 0, Unknown = 1, True = 2 };
    static const int Capacity = 9; // Увеличиваем емкость на 1
        для хранения знака

    LongTritNumber() {
        trits.resize(Capacity, Trit::False);
    }

    const Trit& operator[](size_t index) const {

```

```

        return trits[index];
    }

LongTritNumber& operator=(const LongTritNumber& other) {
    if (this != &other) {
        for (size_t i = 0; i < Capacity; ++i) {
            trits[i] = other.trits[i];
        }
    }
    return *this;
}

bool operator==(const LongTritNumber& other) const {
    if (trits[0] != other.trits[0]) {
        return false;
    }

    for (size_t i = 0; i < Capacity; ++i) {
        if (trits[i] != other.trits[i]) {
            return false;
        }
    }
    return true;
}

bool operator!=(const LongTritNumber& other) const {
    for (size_t i = 0; i < Capacity; ++i) {
        if (trits[i] != other.trits[i]) {
            return true;
        }
    }
    return false;
}

LongTritNumber operator+(const LongTritNumber& other) const {
    LongTritNumber result;
    Trit carry = Trit::False;

    for (int i = Capacity - 1; i >= 0; --i) {
        Trit sum = addTrits(trits[i], other.trits[i],
            carry);
        result.trits[i] = sum;
    }
}

```

```

    }

    return result;
}

LongTritNumber operator-(const LongTritNumber& other) const {
    LongTritNumber result;
    Trit borrow = Trit::False;

    for (int i = Capacity - 1; i >= 0; --i) {
        Trit diff = subtractTrits(trits[i], other.trits[i], borrow);
        result.trits[i] = diff;
        borrow = calculateBorrow(trits[i], other.trits[i], borrow);
    }

    return result;
}

int operator<(const LongTritNumber& other) const {
    if (trits[0] != other.trits[0]) {
        return (trits[0] == Trit::True);
    }
    for (size_t i = 1; i < Capacity; ++i) {
        if (trits[i] < other.trits[i]) {
            return true;
        }
        else if (trits[i] > other.trits[i]) {
            return false;
        }
    }
    return 3;
}

int operator>(const LongTritNumber& other) const {
    if (trits[0] != other.trits[0]) {
        return (trits[0] == Trit::False);
    }
    for (size_t i = 1; i < Capacity; ++i) {
        if (trits[i] > other.trits[i]) {
            return true;
        }
    }
}

```



```

        }
        else if (trits[i] < other.trits[i]) {
            return false;
        }
    }
    return 3;
}

bool operator<=(const LongTritNumber& other) const {
    if (trits[0] != other.trits[0]) {
        return (trits[0] == Trit::True);
    }
    for (size_t i = 1; i < Capacity; ++i) {
        if (trits[i] <= other.trits[i]) {
            return true;
        }
        else if (trits[i] >= other.trits[i]) {
            return false;
        }
    }
    return false;
}

bool operator>=(const LongTritNumber& other) const {
    if (trits[0] != other.trits[0]) {
        return (trits[0] == Trit::False);
    }
    for (size_t i = 1; i < Capacity; ++i) {
        if (trits[i] >= other.trits[i]) {
            return true;
        }
        else if (trits[i] <= other.trits[i]) {
            return false;
        }
    }
    return false;
}

friend ostream& operator<<(ostream& os, const LongTritNumber&
    number) {
    if (number.trits[0] == Trit::True) {
        os << "Отрицательное число: ";
    }
}

```

```

    }
    else {
        os << "Положительное число: ";
    }
    for (size_t i = 1; i < number.trits.size(); ++i) {
        switch (number.trits[i]) {
            case Trit::False:
                os << "False";
                break;
            case Trit::Unknown:
                os << "Unknown";
                break;
            case Trit::True:
                os << "True";
                break;
        }
        os << " ";
    }
    return os;
}

friend istream& operator>>(istream& is, LongTritNumber& number) {
    int sign = 0;
    cout << "Введите знакчисла (0 - положительное / 1 - отрицательное): ";
    is >> sign;

    if (sign != 0 && sign != 1) {
        cout << "Некорректный вводзнакачисла ." << endl;
        return is;
    }

    number.trits[0] = (sign == 1) ? Trit::True : Trit::False;

    for (size_t i = 1; i < number.trits.size(); ++i) {
        int userInput;
        cout << "Введите значения Trita (0 - False, 1 - Unknown, 2 - True): ";
        is >> userInput;

        if (userInput >= 0 && userInput <= 2) {
            number.trits[i] = static_cast<Trit>(

```

```

        userInput);
    }
    else {
        cout << "Некорректный ввод,
                используйте значения от 0 до 2." << endl;
        return is;
    }
}
return is;
}

private:
    vector<Trit> trits;

Trit addTrits(Trit a, Trit b, Trit& carry) const {
    int sum = static_cast<int>(a) + static_cast<int>(b) +
        static_cast<int>(carry);
    carry = static_cast<Trit>(sum / 3);
    return static_cast<Trit>(sum % 3);
}

Trit calculateCarry(Trit a, Trit b, Trit carry) const {
    int sum = static_cast<int>(a) + static_cast<int>(b) +
        static_cast<int>(carry);
    return static_cast<Trit>(sum / 3); // 1 если sum >= 3,
        иначе 0
}

Trit subtractTrits(Trit a, Trit b, Trit& borrow) const {
    int diff = static_cast<int>(a) - static_cast<int>(b) -
        static_cast<int>(borrow);
    if (diff < 0) {
        diff += 3;
        borrow = Trit::True;
    }
    else {
        borrow = Trit::False;
    }
    return static_cast<Trit>(diff);
}

Trit calculateBorrow(Trit a, Trit b, Trit borrow) const {

```

```

        int diff = static_cast<int>(a) - static_cast<int>(b) -
                    static_cast<int>(borrow);
        return (diff < 0) ? Trit::True : Trit::False;
    }
};

int main() {
    setlocale(0, "");
    LongTritNumber a, b;

    cout << "Введите числа:" << endl;
    cout << "Число a:" << endl;
    cin >> a;

    cout << "Число b:" << endl;
    cin >> b;

    LongTritNumber c = a + b;
    LongTritNumber d = a - b;

    cout << endl;
    cout << "Результат сложения: " << c << endl;
    cout << "Результат вычитания: " << d << endl;

    if (a == b) {
        cout << "a равно b" << endl;
    }
    else {
        cout << "a неравно b" << endl;
    }

    if (a != b) {
        cout << "a неравно b" << endl;
    }
    else {
        cout << "a равно b" << endl;
    }

    if (a < b) {
        if ((a < b) == 3) {
            cout << "Числа равны" << endl;
        }
    }
}

```

```

        }
        else {
            cout << "a меньше b" << endl;
        }
    }
    else {
        cout << "a неменьше b" << endl;
    }

    if (a > b) {
        if ((a > b) == 3){
            cout << "Числа равны" << endl;
        }
        else {
            cout << "a больше b" << endl;
        }
    }
    else {
        cout << "a не больше b" << endl;
    }

    if (a <= b) {
        cout << "a меньшеилиравно b" << endl;
    }
    else {
        cout << "a неменьшеилиравно b" << endl;
    }

    if (a >= b) {
        cout << "a большеилиравно b" << endl;
    }
    else {
        cout << "a не большеилиравно b" << endl;
    }

    return 0;
}

```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы.

Результат работы программы, где пользователь вводит троичное число, и программа выводит сумму двух чисел, разность и сравнивает их. Результат приведён на рис. 1.

```
Введите числа:
Число a:
Введите знак числа (0 - положительное / 1 - отрицательное): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Число b:
Введите знак числа (0 - положительное / 1 - отрицательное): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 0
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Результат сложения: Положительное число: Unknown False Unknown False True True True False
Результат вычитания: Положительное число: True Unknown Unknown False False False True
a не равно b
a не равно b
a меньше b
a не больше b
a меньше или равно b
a не больше или равно b
```

Рис. 1 – Пример №1

Результат работы программы, где пользователь вводит троичное число, и программа выводит сумму двух чисел, разность и сравнивает их. Результат приведён на рис. 2.

```

Введите числа:
Число a:
Введите знак числа <0 - положительное / 1 - отрицательное>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 2
Число b:
Введите знак числа <0 - положительное / 1 - отрицательное>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 0
Результат сложения: Положительное число: True True True True True True True True
Результат вычитания: Положительное число: True True True True True True True True
a не равно b
a не равно b
a не меньше b
a больше b
a не меньше или равно b
a больше или равно b

```

Рис. 2 – Пример №2

Результат работы программы, где пользователь неверно вводит троичное число. Программа выводит сумму двух чисел, разность и сравнивает их. Результат приведён на рис. 3.

```

Введите числа:
Число a:
Введите знак числа <0 - положительное / 1 - отрицательное>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 662656
Некорректный ввод, используйте значения от 0 до 2.
Число b:
Введите знак числа <0 - положительное / 1 - отрицательное>: 0
Введите значение трита <0 - False, 1 - Unknown, 2 - True>: 4166654
Некорректный ввод, используйте значения от 0 до 2.
Результат сложения: Положительное число: False False False False False False False False
Результат вычитания: Положительное число: False False False False False False False False
a равно b
Числа равны
a меньше или равно b
a больше или равно b

```

Рис. 3 – Пример №3

Результат работы программы, где пользователь вводит два отрицательных троичных числа. Программа выводит сумму двух чисел, разность и сравнивает их. Результат приведён на рис. 4.

```

Введите числа:
Число a:
Введите знак числа (0 - положительное / 1 - отрицательное): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Число b:
Введите знак числа (0 - положительное / 1 - отрицательное): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 1
Введите значение трита (0 - False, 1 - Unknown, 2 - True): 2
Результат сложения: Отрицательное число: Unknown Unknown Unknown Unknown Unknown Unknown Unknown Fals
е
Результат вычитания: Положительное число: True False True False True False True True
a не равно b
a не равно b
a не меньше b
a больше b
a не меньше или равно b
a больше или равно b

```

Рис. 4 – Пример №4

Результат работы программы, где пользователь неверно вводит знак троичных чисел. Программа выводит сумму двух чисел, разность и сравнивает их. Результат приведён на рис. 5.

```
Введите числа:
Число a:
Введите знак числа <0 – положительное / 1 – отрицательное>: 5
Некорректный ввод знака числа.
Число b:
Введите знак числа <0 – положительное / 1 – отрицательное>: 9
Некорректный ввод знака числа.

Результат сложения: Положительное число: False False False False False False False False
Результат вычитания: Положительное число: False False False False False False False False
a равно b
a равно b
Числа равны
Числа равны
a меньше или равно b
a больше или равно b
```

Рис. 5 – Пример №5

Заключение

Цель лабораторной работы достигнута: создана собственная структура данных.

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описаны троичные числа.
2. Разработана структура данных для хранения целых чисел с повышением ёмкости хранения по сравнению с базовым типом данных.
3. Реализованы 6 операторов сравнения, операторы присваивания (копии значения), сложения и вычитания двух структур длинных чисел.
4. Выполнено тестирование.

Список литературы

1. Рамиль Альварес Х. Алгоритмы троичной арифметики. – М., Фонд «Новое тысячелетие», 2012. – 20 с.